

A Preliminary Study on How AI Coding Agents Describe Refactoring Activities

Aymen Masmoudi*, Belhassen Khelifa*, Andrew Haralambous*,
Eman Abdullah AlOmar†, Mohamed Wiem Mkaouer*

*University of Michigan-Flint, Flint, Michigan, USA

†Stevens Institute of Technology, Hoboken, New Jersey, USA

Email: {masmoudi, khelifa, aharalam, mmkaouer}@umich.edu, ealomar@stevens.edu

Abstract—Refactoring is essential for maintaining and evolving software systems, yet we still have limited insight into how automated code-generation agents describe these changes in pull requests. In this study, we analyze refactoring-related pull requests produced by five AI coding agents, with a particular focus on how their intentions are communicated through pull request descriptions. To better understand recurring transformation patterns, we extract refactoring descriptors terms that capture different refactoring activities and use them to identify similarities across agents. We then classify the pull requests into three main categories: internal quality attributes, external quality attributes, and code smells. Our dataset includes 2,288 unique AI-generated refactoring pull requests from OpenAI Codex, Devin, GitHub Copilot, Cursor, and Claude Code. The results reveal several limitations in how AI systems perform and document refactoring, highlighting the need for clearer and more structured communication in collaborative software development.

Index Terms—Refactoring, Large Language Models, Autonomous Coding Agents, Pull Requests, Software Quality

I. INTRODUCTION

Refactoring plays a crucial role in keeping software systems understandable and adaptable over time [1]. With the growing adoption of AI-based coding agents in development workflows, an increasing number of refactoring changes are now automatically generated and submitted as pull requests (PRs) [2]. These contributions typically aim to reorganize code structure, simplify implementations, or enhance readability [3]. Despite their growing presence in collaborative version-control environments, the way these changes are presented and reviewed has not been thoroughly studied [4]. In particular, it is still unclear how AI agents communicate the intent behind their refactoring actions. As illustrated in Figure 1, PR #10264 in the dotnet/android project removes three unused struct types (BinaryTypeMapHeader, TypeMapIndexHeader, and XamarinAndroidBundledAssembly) from a C++ header file. The PR description explicitly states “no functional changes to the runtime,” and the change amounts to 18 deletions across 2 files. This makes it a clear candidate for the *internal quality attribute* category in our classification scheme, as it targets code maintainability specifically dead code removal without altering any externally observable behavior.

These observations raise important questions about how AI agents describe refactoring changes. To investigate this, we

conduct an empirical study formalized through the following research question:

RQ: *In what ways do AI agents describe and organize refactoring changes in pull requests?* To address this question, we mined textual content from refactoring-related pull requests and grouped them into three distinct categories: internal quality attributes, external quality attributes, and code smells.

To summarize, this paper makes the following contributions:

- We analyze and label 2,288 AI-generated refactoring pull requests (PRs) spanning five widely-used AI agents OpenAI Codex, Devin, GitHub Copilot, Cursor, and Claude Code into three broad classes: Internal Quality Attributes (51.35%), External Quality Attributes (34.62%), and Code Smells (14.03%). These findings reveal the dominant areas where AI agents concentrate their efforts, particularly around enhancing maintainability and readability.
- Our dataset is available for replication and extension purposes¹.

II. STUDY DESIGN

A. Dataset Extraction

To extract agentic PRs, our study draws on the AIDev dataset [5], which consists of pull requests produced by autonomous coding agents working across a diverse set of open-source GitHub repositories. Leveraging this dataset as a start allows us to use a curated set of agentic PRs. To answer our RQ, we go beyond code changes to also capture broader development context, such as PR documentation.

B. Refactoring Motivation Extraction

Since the dataset contains 33,596 pull requests with task categories such as fix, refactor, and documentation, we select pull requests with a task type of refactoring to extract refactoring motivation. From these, we identified 2,288 unique AI-generated pull requests labeled with `type = refactor` and produced by five agents (OpenAI Codex, Devin, Copilot, Cursor, and Claude Code). To address our RQ, we analyze the pull request title and description using the three main motivations behind refactoring: Internal quality attributes, External quality attributes, and Code smells.

¹<https://smilevo.github.io/agentic-refactoring-prs-replication/>

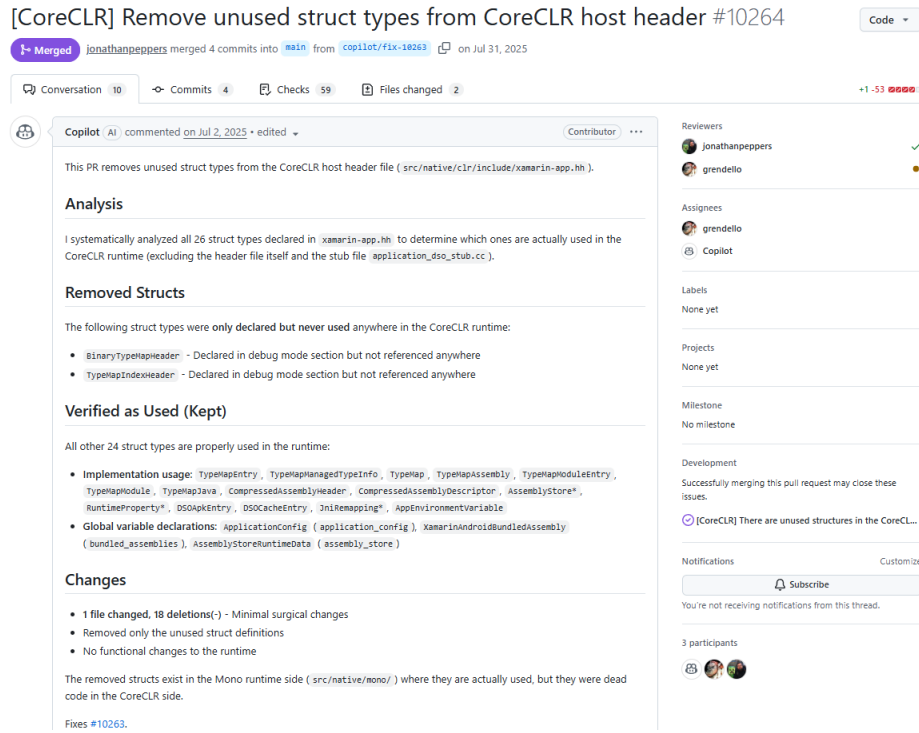


Fig. 1. An Example of agentic refactoring pull request by automated by Copilot

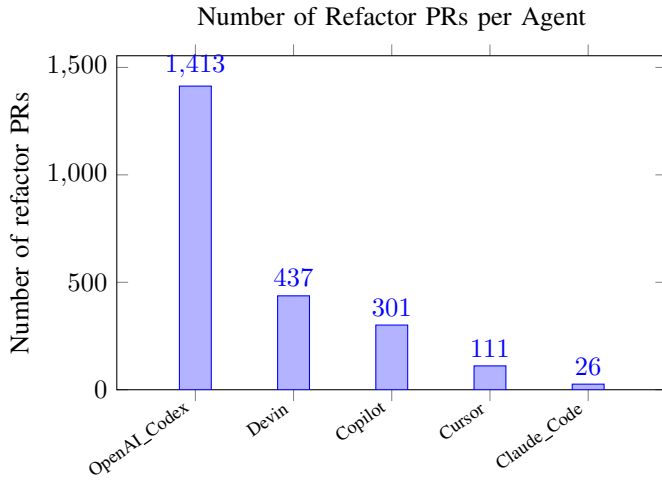


Fig. 2. Distribution of refactor PRs across the five AI agents in the AIDev dataset.

III. EXPERIMENTS

To understand how AI agents document refactoring activity in pull requests, we conducted two complementary analyses. As a first step, we examined the language used in pull request titles and descriptions to identify action-oriented verbs associated with refactoring intent. This analysis revealed a recurring set of verbs that signal common transformation goals, such as restructuring code, improving readability, or

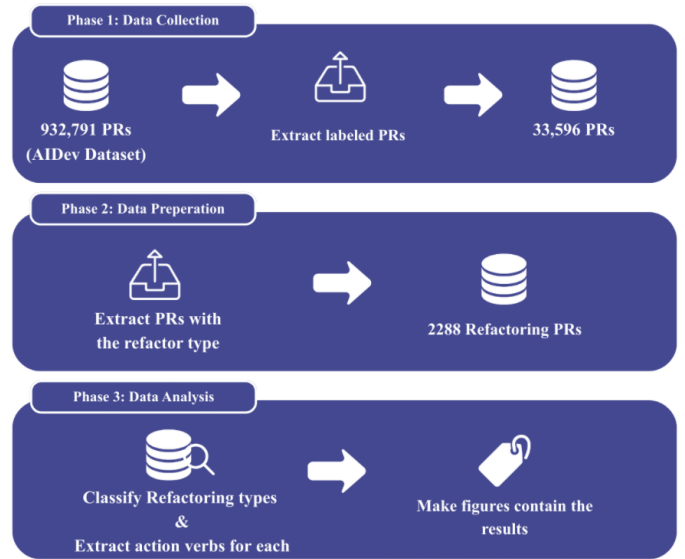


Fig. 3. Study design: Overview of data collection and analysis process, from labeled pull requests to classified refactoring types and extracted refactoring descriptors.

removing unnecessary elements. As shown in Figure 4, generic verbs dominate this vocabulary: *refactor* (2222 occurrences), *fix* (1306), and *improve* (905) appear most frequently, followed by more specific actions such as *move* (515), *remove* (441), *update* (338), and *clean* (267). More fine-grained refactoring

verbs, including *rename* (90), *extract* (24), *split* (25), and *unify* (19), occur considerably less often. This distribution suggests that AI agents tend to describe refactoring using broad, high-level terms rather than precise refactoring operations. In addition, the frequent co-occurrence of multiple refactoring descriptors within a single pull request indicates that agent-generated refactorings are often multifaceted, combining several transformation goals such as restructuring, cleanup, and renaming within the same submission.

Table I explains the meaning of each refactoring descriptor. Second, we performed a data-driven classification of refactoring pull requests based on the recurring terms used in their titles and descriptions. Building on prior refactoring research, we grouped action-oriented keywords into three high-level categories: (1) *Internal Quality Attributes*, such as abstraction, coupling, and dependency management; (2) *External Quality Attributes*, such as readability and maintainability; and (3) *Code Smells*, such as duplicate code. Each pull request was assigned to one or more categories depending on the refactoring-related keywords it contained.

The classification of pull requests is summarized in figure 5,

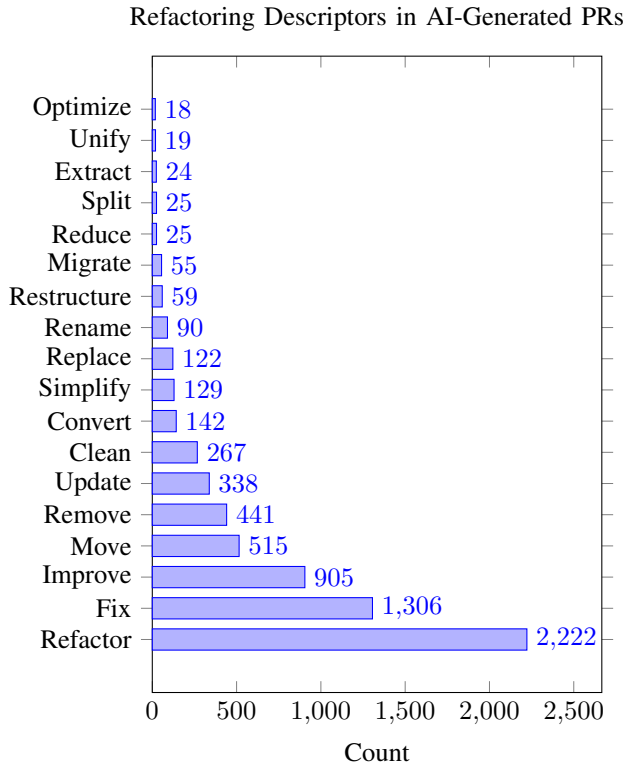


Fig. 4. Distribution of refactoring descriptors in AI-generated PRs.

figure 6 and figure 7.

TABLE I
DEFINITIONS OF REFACTORING REFACTORING DESCRIPTORS USED IN AI-GENERATED PULL REQUESTS

Refactoring Descriptors	Meaning / Typical Change
Refactor	Generic code improvement or reorganization, enhancing structure without altering behavior
Fix	Corrects errors or bugs in the codebase
Improve	Enhances functionality, quality, or code clarity
Move	Relocates code blocks, files, or functions
Remove	Deletes unused or redundant code
Update	Refreshes dependencies, APIs, or documentation
Clean	Cleans up formatting, comments, warnings, or unused code
Convert	Changes data types, structures, or formats
Simplify	Rewrites code to make it easier to read or maintain
Replace	Substitutes old code with new implementation
Rename	Changes names of variables, functions, or classes for clarity
Restructure	Reorganizes code on a larger scale, altering file/module/class structure
Migrate	Transfers code/components to new interfaces, platforms, or versions
Reduce	Minimizes code complexity, size, or resource usage
Split	Divides code/functions into smaller, modular pieces
Extract	Pulls out code blocks to improve modularity or reuse
Unify	Combines duplicated patterns, logic, or code blocks
Optimize	Improves efficiency, performance, or resource usage

TABLE II
DISTRIBUTION OF REFACTOR TYPES ACROSS AI-GENERATED PRs

Refactor Category	Percentage
Internal Quality Attributes	51.35%
External Quality Attributes	34.62%
Code Smells	14.03%

Table II summarizes the distribution of refactoring categories across all 2,288 PRs.

Figure 5 shows the distribution of internal quality attribute refactoring types performed by various AI agents in our dataset. It demonstrates that these agents largely prioritize large-scale structural changes: structural/architectural refactorings dominate across all tools, with OpenAICodex contributing by far the most, followed by Devin and Copilot. API/interface refactorings comprise a distinct secondary cluster, led mostly by Devin, Copilot, and OpenAICodex. Finer-grained internal refactorings, such as changes to dependency management, abstraction, complexity, coupling, encapsulation, or inheritance, occur only infrequently and in small numbers, indicating that current AI agents rarely focus on these more subtle design

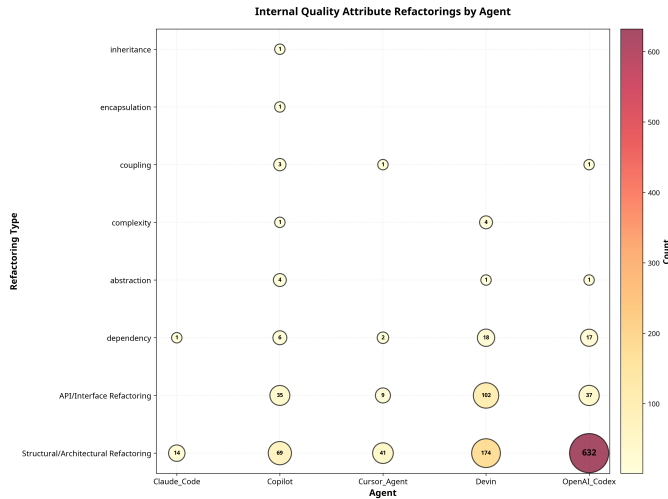


Fig. 5. Distribution of internal quality attribute refactoring types across five AI agents

TABLE III
DESCRIPTION OF INTERNAL QUALITY ATTRIBUTE

Internal attributes	quality	Description
Structural / Architectural Refactoring		Major changes to the overall organization of the codebase, such as reorganizing modules, classes, or files to improve maintainability and scalability.
API / Interface Refactor-ing		Modifications to public APIs or interfaces, including changes in method signatures, data contracts, or endpoints.
Dependency		Adjustments to reduce or clarify module dependencies.
Abstraction		Increasing abstraction or removing duplication.
Complexity		Simplifying complex logic or methods.
Coupling		Reducing tight coupling between components.
Encapsulation		Improving data hiding and state control.
Inheritance		Changes to inheritance hierarchies.

aspects in favor of broader architectural and interface-level improvements. This classification allowed us to compare the refactoring tactics between agents and assess the depth, intent, and quality of the documentation of the resulting refactorings.

The meanings of the internal quality attributes analyzed in this paper are summarized in Table III. Figure 6 illustrates how AI agents refactor external quality attributes, highlighting a significant focus on a few essential problems. Maintainability and readability clearly dominate: across all agents, the majority of external-quality refactorings fall into these two categories, with OpenAICodex accounting for the most, followed by Devin and Copilot. Secondary qualities like compatibility and performance emerge with substantially lower but still apparent frequencies, indicating that some agents, particularly Devin and Cursor, occasionally try to improve how the



Fig. 6. Distribution of external quality attribute refactoring types across five AI agents

system behaves across environments or under pressure. The remaining external properties include accuracy, usability, robustness, accessibility, modularity, dependability, extensibility, and productivity, which occur infrequently and frequently with counts near zero for most agents. This pattern demonstrates that current AI-generated refactorings prioritize making code easier to maintain and comprehend, whereas more specialized external qualities are addressed only occasionally.

The meanings of the external quality attributes analyzed in this paper are summarized in Table IV.

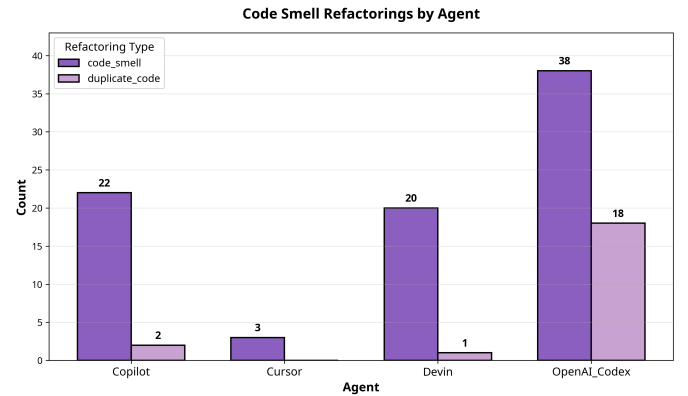


Fig. 7. Code smell refactorings by agent

Figure 7 compares the refactorings related to code smell proposed by several AI agents. The majority of activity is on generic code smells, with OpenAICodex and Copilot contributing the most of these PRs, followed by Devin with a lesser but still substantial share. Duplicate-code refactorings are substantially less common overall, and are nearly exclusively concentrated in OpenAICodex and, to a lesser extent, Copilot, implying that just a few agents routinely handle

TABLE IV
DESCRIPTION OF EXTERNAL QUALITY ATTRIBUTES

External quality attributes	Description
Maintainability	Changes that make the system easier to modify in the future, for example by simplifying code, improving structure, or reducing long-term maintenance effort.
Readability	Refactorings that make code easier for developers to read and understand, such as clearer naming, comments, or formatting.
Compatibility	Adjustments that improve how the system interacts with other components, platforms, or versions (e.g., API or environment compatibility).
Performance	Refactorings aimed at improving execution speed or resource usage, such as optimizing algorithms or reducing unnecessary work.
Correctness	Changes that ensure the system behaves as intended from the user's perspective, often aligning behavior with specifications or fixing externally visible defects.
Usability	Modifications that make the system easier or more intuitive for end users to operate (e.g., clearer messages, simpler workflows).
Robustness	Refactorings that increase the system's ability to handle unexpected situations or invalid inputs without failing.
Accessibility	Changes that improve access for users with diverse needs (e.g., screen-reader support, better contrast or keyboard navigation).
Modularity	Restructuring to create clearer, more independent components, allowing features to be added or changed with minimal side effects.
Reliability	Refactorings that reduce the likelihood of failures in production, for example by improving error handling or fault tolerance.
Extensibility	Changes that make it easier to add new features or extend existing ones, such as adding extension points or more flexible abstractions.
Productivity	Refactorings that speed up development work, for instance by improving build scripts, tooling hooks, or development workflows.

duplication. The meanings of the code smells analyzed in this paper are summarized in Table V.

IV. DISCUSSION AND TAKEAWAYS

The results show that AI-generated pull requests mainly focus on improving the internal structure of code, with most refactorings targeting architectural organization and API-related changes. This suggests that AI agents are more comfortable working on aspects that can be inferred directly from the code itself, such as restructuring files, simplifying layouts, or adjusting interfaces. Improvements to external quality attributes, particularly maintainability and readability, appear mostly as indirect benefits rather than explicit goals, while qualities that depend on runtime behavior or user interaction such as performance or robustness are rarely addressed. Code smells play an important role in motivating refactorings, but AI agents tend to address them in a broad and generic way,

TABLE V
DESCRIPTION OF CODE SMELL

Code smells	Description
code_smell	Generic refactoring aimed at removing or mitigating code smells such as long methods, large classes, unclear responsibilities, or other design problems that harm readability and maintainability.
duplicate_code	Refactoring that removes repeated code fragments by extracting common logic into shared functions, utilities, or abstractions, thereby reducing redundancy and easing future changes.

rather than targeting specific or subtle issues like duplicated logic across files. The refactoring descriptors used in the pull request descriptions further reflect this pattern, emphasizing general intent over precise refactoring operations. Overall, these findings indicate that AI agents act primarily as structural improvers of code, excelling at surface-level quality improvements but remaining limited in addressing deeper design concerns and externally observable software qualities.

V. THREATS TO VALIDITY

Our study is subject to several threats to validity. We relied on the presence of the keyword "refactor" in the titles or bodies of the pull request to identify PR related to refactoring. This method occasionally captured PRs that mentioned refactoring but did not actually perform it. We manually verified and excluded such cases when possible; however, some noise may remain. Second, some pull requests categorized as unmerged at extraction time might have been merged later, as repository states evolve continuously. Finally, we limited our analysis to repositories with at least 500 stars to ensure project maturity, which may restrict generalizability to smaller or less active repositories.

VI. RELATED WORK

Prior work has explored how refactoring activities can be identified and categorized using pull request metadata and natural language. In particular, AlOmar et al. proposed a hybrid taxonomy grounded in Self-Affirmed Refactoring (SAR) patterns and software quality attributes, providing a structured view of refactoring intent expressed in pull requests [6]. Complementing this perspective, recent work by AlOmar et al. [7] examined how refactoring intent is communicated in developer-ChatGPT interactions, offering additional insights into the language used to describe refactoring changes. Together, these studies motivate and shape our approach to analyzing AI-generated refactoring pull requests.

Ottendorf et al. [8] performed an empirical study using AIDev to understand how agents refactor code. Using RefactoringMiner and DesigniteJava, the authors identify refactoring types and detect code smells before and after refactoring commits. Their main findings reveal that agent refactorings are dominated by annotation changes in contrast to the diverse structural improvements typical of developers. Horikawa et

al. [9] conducted an empirical study of GitHub pull requests to examine how often refactoring occurs, the types involved, the purposes, and its effect on quality. Oueslati et al. [10] proposed a multi-agent LLM-based framework for end-to-end refactoring, achieving a median unit-test pass rate of 90%, reducing code smells by 52.5%, and improving key quality attributes by 8.6%. Nakashima et al. [11] performed an empirical study to analyze rejection modes in agentic PRs, finding that a large proportion of rejected PRs (67.9%) lack explicit reviewer feedback. In a separate work, Dilhara et al. [12] outlined best practices for leveraging LLMs to generate code variants and implemented these guidelines in a tool named PYCRAFT. Their findings show that PYCRAFT achieved an F-measure of 96.6% in identifying correct variants. Shirafuji et al. [13] explored the use of LLMs for automated code refactoring. They introduced a technique based on few-shot prompting, in which the LLM is supplied with carefully chosen refactoring examples to improve its ability to transform code. Gao et al. [14] proposed UTRFACTOR, a new framework that employs LLMs together with a Domain-Specific Language-based refactoring strategy to automatically refactor unit tests in Java projects. Zhang et al. [15] tackled the problem of transforming non-idiomatic Python code into idiomatic Pythonic code using a hybrid approach that combines rule-based techniques with LLMs. They highlight three key limitations of earlier refactoring techniques: missed code, incorrect detection, and incorrect refactoring. Liu et al. [16] examined the effectiveness of LLMs in automated software refactoring, assessing how well LLMs can detect refactoring opportunities and propose refactoring solutions in comparison with human experts.

VII. CONCLUSION

This paper presented an empirical study of 2,288 AI-generated refactoring pull requests produced by five coding agents across real-world projects. By combining task-type labels, refactoring descriptors, and a taxonomy of internal, external, and code-smell refactorings, the analysis showed that current agents overwhelmingly focus on large-scale structural and architectural changes, as well as external qualities such as maintainability and readability, while subtler internal attributes (e.g., abstraction, complexity, coupling) and specialized external qualities (e.g., robustness, accessibility, performance) remain rarely targeted. The results suggest that AI assistants are currently most effective for broad cleanup and readability-oriented refactorings, but provide limited support for nuanced design improvements or targeted quality attributes, underscoring the need for improved tooling and guidance in these areas.

ACKNOWLEDGMENTS

This research is funded by the National Science Foundation under Grant No. CNS-2525507.

We declare the usage of OpenAI ChatGPT and Grammarly AI for text editing and LaTeX formatting assistance in drafting this paper.

REFERENCES

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.
- [2] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Pinto, J. Kaplan, H. Edwards, et al., “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [3] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, “A survey of machine learning for big code and naturalness,” *ACM Computing Surveys*, vol. 51, no. 4, p. 81, 2018.
- [4] C. Bird, P. C. Rigby, E. T. Barr, D. J. Hamilton, D. M. German, and P. Devanbu, “The art and science of code review,” *IEEE Software*, vol. 32, no. 6, pp. 85–90, 2015.
- [5] H. Li, H. Zhang, and A. E. Hassan, “The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering,” *arXiv preprint arXiv:2507.15003*, 2025.
- [6] E. AlOmar, M. W. Mkaouer, and A. Ouni, “Can refactoring be self-affirmed? an exploratory study on how developers document their refactoring activities in commit messages,” in *2019 IEEE/ACM 3rd International Workshop on Refactoring (IWor)*, pp. 51–58, IEEE, 2019.
- [7] E. A. AlOmar, A. Venkatakrishnan, M. W. Mkaouer, C. Newman, and A. Ouni, “How to refactor this code? an exploratory study on developer-chatgpt refactoring conversations,” in *Proceedings of the 21st International Conference on Mining Software Repositories*, pp. 202–206, 2024.
- [8] L. Ottenhof, D. Penner, A. Hindle, and T. Lutellier, “How do agents refactor: An empirical study,” *arXiv preprint arXiv:2601.20160*, 2026.
- [9] K. Horikawa, H. Li, Y. Kashiwa, B. Adams, H. Iida, and A. E. Hassan, “Agentic refactoring: An empirical study of ai coding agents,” *arXiv preprint arXiv:2511.04824*, 2025.
- [10] K. Oueslati, M. Lamothe, and F. Khomh, “Refagent: A multi-agent llm-based framework for automatic software refactoring,” *arXiv preprint arXiv:2511.03153*, 2025.
- [11] S. Nakashima, Y. Ishimoto, M. Kondo, S. McIntosh, and Y. Kamei, “Why agentic-prs get rejected: A comparative study of coding agents,” *arXiv preprint arXiv:2602.04226*, 2026.
- [12] M. Dilhara, A. Bellur, T. Bryksin, and D. Dig, “Unprecedented code change automation: The fusion of llms and transformation by example,” *arXiv preprint arXiv:2402.07138*, 2024.
- [13] A. Shirafuji, Y. Oda, J. Suzuki, M. Morishita, and Y. Watanobe, “Refactoring programs using large language models with few-shot examples,” *arXiv preprint arXiv:2311.11690*, 2023.
- [14] Y. Gao, X. Hu, X. Yang, and X. Xia, “Context-enhanced llm-based framework for automatic test refactoring,” *arXiv preprint arXiv:2409.16739*, 2024.
- [15] Z. Zhang, Z. Xing, X. Ren, Q. Lu, and X. Xu, “Refactoring to pythonic idioms: A hybrid knowledge-driven approach leveraging large language models,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1107–1128, 2024.
- [16] B. Liu, Y. Jiang, Y. Zhang, N. Niu, G. Li, and H. Liu, “An empirical study on the potential of llms in automated software refactoring,” *arXiv preprint arXiv:2411.04444*, 2024.